Great Place To Work. Certified FEB 2023 – FEB 2024 INDIA

UPES

UNIVERSITY OF TOMORROW

nirf RANKING 2025

*RANKED 45 IN INDIA

#University Category

QS WORLD UNIVERSITY RANKINGS

NO.1 PVT. UNIVERSITY IN ACADEMIC REPUTATION IN INDIA

NAAC

ACCREDITED **GRADE 'A'** BY NAAC

PERFECT SCORE OF **150/150** AS A TESTAMENT TO EXECEPTIONAL E-LEARNING METHODS

UPES UNIVERSITY OF TOMORROW

Applied Machine Learning

Unit 06 – Lecture 15: Bayesian Algorithms

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

The classifier you can derive from one line of probability theory

Topic focus: **a 99% accurate test, and a 16.67% answer**

Repository: `https://github.com/tali7c/Applied-Machine-Learning`

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 8–9.;James et al., *An Introduction to Statistical Learning with Python*, Ch. 8–9.;Bishop, *Pattern Recognition and Machine Learning*, Ch. 4–5, 7, 14.;Zhang et al., *Dive into Deep Learning*, Ch. 5

Quick Links

Agenda

Bayes' theorem
Two decision rules: MAP and ML
From Bayes' theorem to a classifier
Naive Bayes worked entirely by hand
The zero-frequency problem
Text, log-probabilities and underflow
Gaussian naive Bayes: continuous features
A good classifier and a bad probability estimator
Where the independence assumption genuinely hurts
Summary

How this lecture works

Every idea appears twice

1. **By hand** — one test result, fourteen weather records, four words in an e-mail, one petal width. Small enough to check on paper, and small enough to appear in a test.
2. **In code** — the same arithmetic through `CategoricalNB`, `MultinomialNB` and `GaussianNB`, agreeing to eleven decimal places.

Commit before you look

Five slides ask you to write a number down before it is revealed. **The first one is the whole lecture.** Almost everybody gets it wrong, and being wrong on paper is how the idea sticks.

Two numbers to carry out: a positive result on a 99%-sensitive test means **a** 16.67% **chance of disease**; and a product of 148 probabilities is, in `float64`, **exactly** 0.0.

Four datasets, one seed — and one thing a seed cannot

The data

scikit-learn's bundled `load_iris`, `load_wine`, `load_breast_cancer` and `load_digits`; a twelve-message spam corpus typed out in the source; synthetic documents drawn from the seed. Nothing is downloaded.

The randomness

One seed: `np.random.default_rng(0)` and `random_state=0`. Every listing that draws or splits shows it. Where the *spread* is the measurement — twenty document corpora, thirty repeated splits — the seeds run $0, 1, 2, \ldots$ from that same one.

Except the fit timings

Milliseconds belong to a processor, not to an algorithm. The claim that naive Bayes is cheap to fit is made as a **pass count** — one pass against 15–32 solver iterations — which is true on every machine.

Where we are

Two classifiers so far

k-NN stored the data and voted. A decision tree cut the space with questions. **Neither told you how sure it was**, in any sense you could defend.

Today: start from probability

We will not invent a rule and then measure it. We will write down what we want — $P(\text{class} \mid \text{evidence})$ — and *derive* the classifier from one line of algebra.

The pay-off

Naive Bayes trains in milliseconds, needs almost no data, is the historical spam filter, and is still the first thing to try on text. On `wine` it scores 0.9719.

And the warning

It is a **good classifier and a bad probability estimator**. On `breast_cancer` it claims an average confidence of **0.9936** while getting **0.9342** right. We will measure that.

One line of algebra, and it is the whole lecture

The product rule, written twice

$$P(A, B) = P(A \mid B)\, P(B) \qquad \text{and} \qquad P(A, B) = P(B \mid A)\, P(A)$$

The left-hand sides are the same number, so the right-hand sides are equal. Divide by $P(B)$:

$$\boxed{P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}}$$

That is Bayes' theorem, and **it is not an assumption** — it is the definition of conditional probability rearranged. The content is entirely in what you put in it. *Reproduce this derivation in the examination; it is two lines.*

The four words you must be able to point at

$$\underbrace{P(c \mid x)}_{\text{posterior}} = \frac{\overbrace{P(x \mid c)}^{\text{likelihood}}\ \overbrace{P(c)}^{\text{prior}}}{\underbrace{P(x)}_{\text{evidence}}}$$

Name	Symbol	Read it as
prior	$P(c)$	what I believed about the class *before* seeing this instance
likelihood	$P(x \mid c)$	how well class c explains the evidence I actually saw
evidence	$P(x)$	how likely that evidence was overall $= \sum_{c'} P(x \mid c')P(c')$
posterior	$P(c \mid x)$	what I believe *after* seeing it — the thing I want

The evidence is the same for every class, so if all you want is the *argmax* you may drop it and compare $P(x \mid c)P(c)$. You need it only when you want a number that sums to one.

Predict first #1 — the one that matters

A screening test for a disease

- **Prevalence**: 1% of the population has it. $P(D) = 0.01$.
- **Sensitivity**: if you have it, the test says positive 99% of the time. $P(+ \mid D) = 0.99$.
- **Specificity**: if you do not, the test says negative 95% of the time. $P(- \mid \neg D) = 0.95$, so $P(+ \mid \neg D) = 0.05$.

You test positive. What is the probability that you have the disease?

Write a number down. Do not say it out loud yet.

Predict first #1 — the one that matters

A screening test for a disease

- **Prevalence**: 1% of the population has it. $P(D) = 0.01$.
- **Sensitivity**: if you have it, the test says positive 99% of the time. $P(+ \mid D) = 0.99$.
- **Specificity**: if you do not, the test says negative 95% of the time. $P(- \mid \neg D) = 0.95$, so $P(+ \mid \neg D) = 0.05$.

You test positive. What is the probability that you have the disease?

Write a number down. Do not say it out loud yet.

$P(D \mid +) = \mathbf{0.1667}$ — about **one in six**.

By hand: three lines of arithmetic

Two ways to get a positive result. Add them for the evidence.

$$P(+, D) = 0.99 \times 0.01 = 0.009900$$
$$P(+, \neg D) = 0.05 \times 0.99 = 0.049500$$
$$P(+) = 0.009900 + 0.049500 = 0.059400$$

$$P(D \mid +) = \frac{0.009900}{0.059400} = \frac{99}{594} = \frac{1}{6} = 0.166667$$

Why intuition fails

The test is 99% sensitive, so the mind reaches for 99%. But the question asked about $P(D \mid +)$ and the test specifies $P(+ \mid D)$. **Those are different numbers, and confusing them is called the base-rate fallacy.**

The prior did the work

There are 99 healthy people for every ill one, and the test fires wrongly on 5% of them.

The same thing in whole people — no fractions at all

Take 10,000 people and let the percentages run.

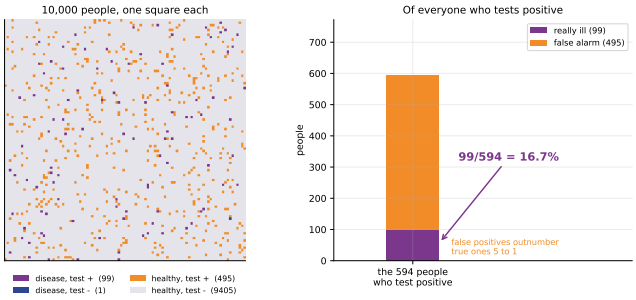
	test +	**test −**	**total**
have the disease	**99**	1	100
do not have it	**495**	9405	9900
total	**594**	9406	10000

594 people are told they are positive. 99 **of them are ill.**

$$\frac{99}{594} = 0.166667$$

False positives outnumber true positives five to one — because a small percentage of a big group beats a big percentage of a small group. Teach yourself to reach for the table, not the formula.

Drawn: ten thousand people, one square each



Purple: ill and correctly flagged, 99 of them. Orange: healthy and wrongly flagged, 495. **The orange squares are what you cannot see when you only look at the test's accuracy figures** — and they are five times as numerous as the purple ones.

And confirmed in code, two ways

```
PREV, SENS, SPEC = 0.01, 0.99, 0.95
num = SENS * PREV # P(+, D)
den = SENS * PREV + (1 - SPEC) * (1 - PREV) # P(+)
print(num / den)
```

```
joint  P(+, D)    = P(+|D) P(D)       = 0.99 * 0.01 = 0.009900
joint  P(+, notD) = P(+|notD) P(notD) = 0.05 * 0.99 = 0.049500
evidence P(+)     = 0.009900 + 0.049500 = 0.059400
POSTERIOR P(D|+)  = 0.009900 / 0.059400 = 0.166667   = 16.6667%
exact fraction    = 99/594 = 1/6 ?  True
```

And confirmed in code, two ways

```
PREV, SENS, SPEC = 0.01, 0.99, 0.95
num = SENS * PREV # P(+, D)
den = SENS * PREV + (1 - SPEC) * (1 - PREV) # P(+)
print(num / den)
```

```
joint  P(+, D)    = P(+|D) P(D)       = 0.99 * 0.01 = 0.009900
joint  P(+, notD) = P(+|notD) P(notD) = 0.05 * 0.99 = 0.049500
evidence P(+)     = 0.009900 + 0.049500 = 0.059400
POSTERIOR P(D|+)  = 0.009900 / 0.059400 = 0.166667   = 16.6667%
exact fraction    = 99/594 = 1/6 ?  True
```

```
confirmed by simulation, 2,000,000 people, seed 0
   simulated P(D|+) = 0.165611   analytic 0.166667   difference 0.001055
```

And confirmed in code, two ways

```
PREV, SENS, SPEC = 0.01, 0.99, 0.95
num = SENS * PREV # P(+, D)
den = SENS * PREV + (1 - SPEC) * (1 - PREV) # P(+)
print(num / den)
```

```
joint  P(+, D)     = P(+|D) P(D)       = 0.99 * 0.01 = 0.009900
joint  P(+, notD)  = P(+|notD) P(notD) = 0.05 * 0.99 = 0.049500
evidence P(+)      = 0.009900 + 0.049500 = 0.059400
POSTERIOR P(D|+)   = 0.009900 / 0.059400 = 0.166667   = 16.6667%
exact fraction     = 99/594 = 1/6 ?  True
```

```
confirmed by simulation, 2,000,000 people, seed 0
   simulated P(D|+) = 0.165611   analytic 0.166667   difference 0.001055
```

Two million simulated people agree with the algebra to 0.001. **The result is not a trick of the formula; it is what actually happens to the population.**

What would make the test useful?

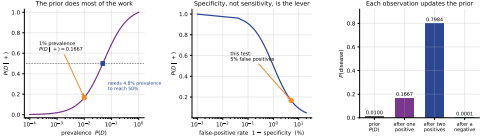
```
prevalence  P(D|+)
    0.001   0.0194
    0.010   0.1667
    0.050   0.5103
    0.200   0.8319
    0.500   0.9519
specificity P(D|+)   (prev. 1%)
    0.9000  0.0909
    0.9500  0.1667
    0.9900  0.5000
    0.9990  0.9091
    0.9999  0.9901
```

Raise the prior

Test people with symptoms rather than everybody. At 20% prevalence the same test gives 0.8319. **Screening a population is a different problem from testing a patient who walked in complaining.**

Or raise the *specificity* — not the sensitivity

95% → 99.9% specificity takes the posterior from 0.1667 to **0.9091**. **On a rare condition, false positives are the entire problem.**



You need 4.8% prevalence before a positive result is an even-money bet. Right-hand panel: **yesterday's posterior is today's prior** — test twice and 0.1667 becomes 0.7984; a negative takes it to 0.0001.

Two rules, one piece of evidence, opposite answers

Maximum likelihood (ML)

$\hat{c}_{\text{ML}} = \arg\max_c P(x \mid c)$ — "which class explains the evidence best?" Ignores how common the classes are.

Maximum a posteriori (MAP)

$\hat{c}_{\text{MAP}} = \arg\max_c P(x \mid c)P(c)$ — "which class is most probable?" This is what a classifier uses. ($P(x)$ is dropped: it cannot change the winner.)

```
MAP against ML on the same evidence
   ML  rule: argmax P(+|c)      -> P(+|D)=0.9900 vs P(+|notD)=0.0500  -> says DISEASE
   MAP rule: argmax P(+|c)P(c)  -> 0.009900 vs 0.049500  -> says NO DISEASE
   the prior ratio is 99:1 against; the likelihood ratio is only 19.8:1 in favour
```

Two rules, one piece of evidence, opposite answers

Maximum likelihood (ML)

$\hat{c}_{\text{ML}} = \arg\max_c P(x \mid c)$ — "which class explains the evidence best?" Ignores how common the classes are.

Maximum a posteriori (MAP)

$\hat{c}_{\text{MAP}} = \arg\max_c P(x \mid c)P(c)$ — "which class is most probable?" This is what a classifier uses. ($P(x)$ is dropped: it cannot change the winner.)

```
MAP against ML on the same evidence
   ML  rule: argmax P(+|c)      -> P(+|D)=0.9900 vs P(+|notD)=0.0500  -> says DISEASE
   MAP rule: argmax P(+|c)P(c)  -> 0.009900 vs 0.049500  -> says NO DISEASE
   the prior ratio is 99:1 against; the likelihood ratio is only 19.8:1 in favour
```

Read the last line slowly

The evidence is 19.8 times more consistent with disease than with health. That is genuinely strong evidence. **It is not strong enough**, because it starts 99 to 1 behind. $19.8 < 99$, so the posterior odds are still against.

Two rules, one piece of evidence, opposite answers

Maximum likelihood (ML)

$\hat{c}_{\text{ML}} = \arg\max_c P(x \mid c)$ — "which class explains the evidence best?" Ignores how common the classes are.

Maximum a posteriori (MAP)

$\hat{c}_{\text{MAP}} = \arg\max_c P(x \mid c)P(c)$ — "which class is most probable?" This is what a classifier uses. ($P(x)$ is dropped: it cannot change the winner.)

```
MAP against ML on the same evidence
   ML  rule: argmax P(+|c)      -> P(+|D)=0.9900 vs P(+|notD)=0.0500  -> says DISEASE
   MAP rule: argmax P(+|c)P(c)  -> 0.009900 vs 0.049500  -> says NO DISEASE
   the prior ratio is 99:1 against; the likelihood ratio is only 19.8:1 in favour
```

Read the last line slowly

The evidence is 19.8 times more consistent with disease than with health. That is genuinely strong evidence. **It is not strong enough**, because it starts 99 to 1 behind. $19.8 < 99$, so the posterior odds are still against.

Posterior odds = likelihood ratio $\times$ prior odds: $\frac{1}{5} = 19.8 \times \frac{1}{99}$. **One multiplication, and it is the whole of Bayesian updating.**

Predict first #2

An instance is a vector $x = (x_1, \ldots, x_d)$, so $\hat{c} = \arg\max_c P(c)\, P(x_1, \ldots, x_d \mid c)$. But $P(x_1, \ldots, x_d \mid c)$ is a probability for *every combination* of feature values: with d binary features, a table of 2^d cells per class, $C\,(2^d - 1)$ free parameters, every one estimated from data.

At $d = 30$ — fewer features than `breast_cancer` has — how many is that? And how many rows do you have?

Predict first #2

An instance is a vector $x = (x_1, \ldots, x_d)$, so $\hat{c} = \arg\max_c P(c)\, P(x_1, \ldots, x_d \mid c)$. But $P(x_1, \ldots, x_d \mid c)$ is a probability for *every combination* of feature values: with d binary features, a table of 2^d cells per class, $C\,(2^d - 1)$ free parameters, every one estimated from data.

At $d = 30$ — fewer features than `breast_cancer` has — how many is that? And how many rows do you have?

```
d binary features, 2 classes
    d        joint  C(2^d - 1)     naive  C*d            ratio
    3                       14         6                    2
    5                       62        10                    6
   10                    2,046        20                  102
   20                2,097,150        40               52,429
   30            2,147,483,646        60           35,791,394
   64 36,893,488,147,419,103,230       128 288,230,376,151,711,744
```

Predict first #2

An instance is a vector $x = (x_1, \ldots, x_d)$, so $\hat{c} = \arg\max_c P(c)\, P(x_1, \ldots, x_d \mid c)$. But $P(x_1, \ldots, x_d \mid c)$ is a probability for *every combination* of feature values: with d binary features, a table of 2^d cells per class, $C\,(2^d - 1)$ free parameters, every one estimated from data.

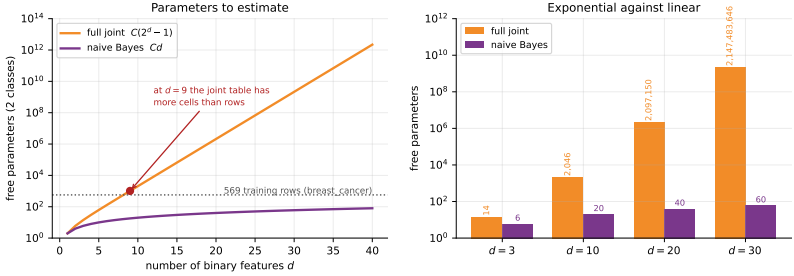
At $d = 30$ — fewer features than `breast_cancer` has — how many is that? And how many rows do you have?

```
d binary features, 2 classes
    d        joint  C(2^d - 1)      naive  C*d            ratio
    3                       14          6                2
    5                       62         10                6
   10                    2,046         20              102
   20                2,097,150         40           52,429
   30            2,147,483,646         60       35,791,394
   64 36,893,488,147,419,103,230        128 288,230,376,151,711,744
```

Two billion parameters, five hundred and sixty-nine rows

`breast_cancer` has 569 patients. The joint table at $d = 30$ has 2,147,483,646 free parameters. **You have about one row for every four million parameters.** At $d = 64$ — `load_digits`, one bit per pixel — the joint table has more cells than there are grains of sand on Earth.

Drawn: exponential against linear



The joint table passes the 569 available training rows at $d = 9$. Nine binary features. **Everything past that point is a table you cannot fill**, and the naive line — the purple one — is the only thing on the plot that stays inside your data budget.

The naive assumption, stated precisely

Conditional independence given the class

$$P(x_1, x_2, \ldots, x_d \mid c) \;=\; \prod_{j=1}^{d} P(x_j \mid c)$$

Equivalently: *once you know the class*, knowing one feature tells you nothing about any other.
$P(x_i \mid c, x_j) = P(x_i \mid c)$ for all $i \neq j$.

What it does not say	**What it does say**
the features are independent	they are independent *within each class*
outlook and *humidity* are unrelated	they are unrelated *among the days you played*

It is almost always false. It is made anyway, because it is the difference between 2^d parameters and d of them — and because, as we shall measure, the resulting classifier is often right even when its probabilities are not.

What the assumption buys, on one concrete table

Fourteen weather records. `Outlook` has 3 levels, `Temperature` 3, `Humidity` 2, `Wind` 2; two classes.

```
the tennis table: outlook(3) temperature(3) humidity(2) wind(2), 2 classes
   full joint        70 free parameters (a 36-cell table per class)
   naive Bayes       12 free parameters + 1 for the prior
   training rows available: 14
   rows per joint cell on average: 0.194
```

What the assumption buys, on one concrete table

Fourteen weather records. `Outlook` has 3 levels, `Temperature` 3, `Humidity` 2, `Wind` 2; two classes.

```
the tennis table: outlook(3) temperature(3) humidity(2) wind(2), 2 classes
   full joint        70 free parameters (a 36-cell table per class)
   naive Bayes       12 free parameters + 1 for the prior
   training rows available: 14
   rows per joint cell on average: 0.194
```

The joint way

$3 \times 3 \times 2 \times 2 = 36$ cells per class, 70 free parameters in all. With 14 rows you get 0.194 **rows per cell**: most cells are empty and most probabilities are 0.

The naive way

$(3-1)+(3-1)+(2-1)+(2-1) = 6$ per class, so **12** plus one prior. **Every one of them is estimated from at least five rows.** That is the trade: a wrong model you can fit against a right model you cannot.

The naive Bayes classifier, complete

$$\hat{c} \;=\; \arg\max_{c \in \mathcal{C}} \; P(c) \prod_{j=1}^{d} P(x_j \mid c)$$

Quantity	Estimated as	Called
$P(c)$	N_c / N	the class prior
$P(x_j \mid c)$, categorical	$N_{jc}(x_j)/N_c$	`CategoricalNB`
$P(x_j \mid c)$, word counts	$(T_{cw})/\sum_{w'} T_{cw'}$	`MultinomialNB`
$P(x_j \mid c)$, continuous	$\mathcal{N}(x_j; \mu_{jc}, \sigma^2_{jc})$	`GaussianNB`

Training is counting. There is no iteration, no gradient, no hyperparameter to search except the smoothing constant. On `digits` it fits in 1.23 ms against logistic regression's 127.78 ms.

The fourteen rows

#	Outlook	Temperature	Humidity	Wind	**Play**
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Nine *Yes*, five *No*. Priors $P(\text{Yes}) = 9/14 = 0.642857$ and $P(\text{No}) = 5/14 = 0.357143$. **This is the table every Indian syllabus uses; learn it once.**

Step 1: the tables — which is just counting

Feature	Level	given Yes ($N = 9$)		given No ($N = 5$)	
Outlook	Overcast	4/9	$= 0.4444$	**0/5**	$=$ **0.0000**
	Rain	3/9	$= 0.3333$	2/5	$= 0.4000$
	Sunny	2/9	$= 0.2222$	3/5	$= 0.6000$
Temperature	Cool	3/9	$= 0.3333$	1/5	$= 0.2000$
	Hot	2/9	$= 0.2222$	2/5	$= 0.4000$
	Mild	4/9	$= 0.4444$	2/5	$= 0.4000$
Humidity	High	3/9	$= 0.3333$	4/5	$= 0.8000$
	Normal	6/9	$= 0.6667$	1/5	$= 0.2000$
Wind	Strong	3/9	$= 0.3333$	3/5	$= 0.6000$
	Weak	6/9	$= 0.6667$	2/5	$= 0.4000$

Each block sums to 1 down its column. **That is the entire training procedure.** Note the bold zero — we come back to it in ten minutes.

Step 2: classify a new day, by hand

New instance: $x = (\text{Sunny}, \text{Cool}, \text{High}, \text{Strong})$.

Score for *No*

$$\tfrac{5}{14} \times \tfrac{3}{5} \times \tfrac{1}{5} \times \tfrac{4}{5} \times \tfrac{3}{5}$$
$$= 0.357143 \times 0.6 \times 0.2 \times 0.8 \times 0.6$$
$$= \mathbf{0.02057143}$$

Score for *Yes*

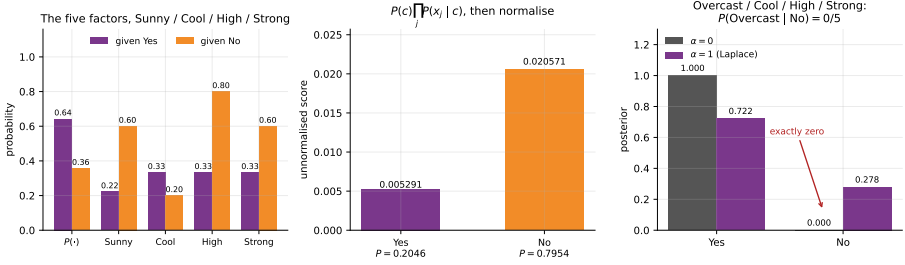
$$\tfrac{9}{14} \times \tfrac{2}{9} \times \tfrac{3}{9} \times \tfrac{3}{9} \times \tfrac{3}{9}$$
$$= 0.642857 \times 0.2222 \times 0.3333^3$$
$$= \mathbf{0.00529101}$$

Evidence $P(x) = 0.02057143 + 0.00529101 = 0.02586243$, so

$$P(\text{No} \mid x) = \frac{0.02057143}{0.02586243} = \mathbf{0.795417}, \qquad P(\text{Yes} \mid x) = \mathbf{0.204583}.$$

Prediction: No, at posterior odds 3.8880 : 1. Five multiplications and one division.

Drawn: the five factors and their product



Left: the five numbers that get multiplied. *High* humidity is the strongest single piece of evidence against playing (0.80 against 0.33). Centre: their products, before normalising. Right: what happens when one factor is 0/5 — the next section.

The identical result from scikit-learn

```
enc = OrdinalEncoder(categories=LEVELS)
Xe = enc.fit_transform(np.array(X_t, dtype=object))
cnb0 = CategoricalNB(alpha=1e-10, force_alpha=True).fit(Xe, ye)
cnb0.predict_proba(enc.transform([["Sunny","Cool","High","Strong"]]))
```

```
the same instance through sklearn CategoricalNB(alpha=1e-10)
      predict_proba  No 0.795417   Yes 0.204583
      predict        No
      agrees with the hand calculation to 7.68e-12
```

The identical result from scikit-learn

```
enc = OrdinalEncoder(categories=LEVELS)
Xe = enc.fit_transform(np.array(X_t, dtype=object))
cnb0 = CategoricalNB(alpha=1e-10, force_alpha=True).fit(Xe, ye)
cnb0.predict_proba(enc.transform([["Sunny","Cool","High","Strong"]]))
```

```
the same instance through sklearn CategoricalNB(alpha=1e-10)
      predict_proba  No 0.795417   Yes 0.204583
      predict        No
      agrees with the hand calculation to 7.68e-12
```

```
resubstitution accuracy of the hand model on all 14 rows
      13/14 = 0.9286
```

The identical result from scikit-learn

```
enc = OrdinalEncoder(categories=LEVELS)
Xe = enc.fit_transform(np.array(X_t, dtype=object))
cnb0 = CategoricalNB(alpha=1e-10, force_alpha=True).fit(Xe, ye)
cnb0.predict_proba(enc.transform([["Sunny","Cool","High","Strong"]]))
```

```
the same instance through sklearn CategoricalNB(alpha=1e-10)
      predict_proba  No 0.795417   Yes 0.204583
      predict        No
      agrees with the hand calculation to 7.68e-12
```

```
resubstitution accuracy of the hand model on all 14 rows
      13/14 = 0.9286
```

There is nothing inside `CategoricalNB` **that you did not just do with a pen.** Note $\alpha = 10^{-10}$: the library smooths by default, and to reproduce the textbook arithmetic you must turn it off. In a moment you will see why the default exists.

One empty cell kills the whole product

Look again at the table: **Overcast never appears with No**. Four overcast days, all of them played.

$$P(\text{Overcast} \mid \text{No}) = \frac{0}{5} = 0.$$

Now classify $x = (\text{Overcast}, \text{Cool}, \text{High}, \text{Strong})$. Three of the four features favour *No*: *High* humidity (0.80 against 0.33), *Strong* wind (0.60 against 0.33), *Cool* is near-neutral.

$$\text{Score for } \textit{No}: \quad \frac{5}{14} \times \frac{\mathbf{0}}{\mathbf{5}} \times \frac{1}{5} \times \frac{4}{5} \times \frac{3}{5}$$

One empty cell kills the whole product

Look again at the table: **Overcast never appears with No**. Four overcast days, all of them played.

$$P(\text{Overcast} \mid \text{No}) = \frac{0}{5} = 0.$$

Now classify $x = (\text{Overcast}, \text{Cool}, \text{High}, \text{Strong})$. Three of the four features favour *No*: *High* humidity (0.80 against 0.33), *Strong* wind (0.60 against 0.33), *Cool* is near-neutral.

$$\text{Score for } \textit{No}\text{: } \frac{5}{14} \times \frac{\mathbf{0}}{\mathbf{5}} \times \frac{1}{5} \times \frac{4}{5} \times \frac{3}{5}$$

Predict first #3: what is that product?

0.00000000, so $P(\text{No} \mid x)$ is **exactly zero**.

"Impossible", on the evidence of four missing rows

```
classify ('Overcast', 'Cool', 'High', 'Strong') with alpha = 0
      P(No) * prod = 5/14 * 0/5 * 1/5 * 4/5 * 3/5 = 0.00000000
      P(Yes) * prod = 9/14 * 4/9 * 3/9 * 3/9 * 3/9 = 0.01058201
      P(Yes|x) = 1.000000   P(No|x) = 0.000000
      the other three factors for No were 1/5, 4/5, 3/5 -- none of them small.
```

"Impossible", on the evidence of four missing rows

```
classify ('Overcast', 'Cool', 'High', 'Strong') with alpha = 0
      P(No) * prod = 5/14 * 0/5 * 1/5 * 4/5 * 3/5 = 0.00000000
      P(Yes) * prod = 9/14 * 4/9 * 3/9 * 3/9 * 3/9 = 0.01058201
      P(Yes|x) = 1.000000   P(No|x) = 0.000000
      the other three factors for No were 1/5, 4/5, 3/5 -- none of them small.
```

A probability of exactly zero is a claim you cannot defend

The model is not saying "unlikely". It is saying **impossible**: no amount of evidence from the other three features can ever recover it, because they are all multiplied by 0. **One unobserved combination has vetoed the entire class.** And the sample size behind that veto is *five rows*.

"Impossible", on the evidence of four missing rows

```
classify ('Overcast', 'Cool', 'High', 'Strong') with alpha = 0
      P(No) * prod = 5/14 * 0/5 * 1/5 * 4/5 * 3/5 = 0.00000000
      P(Yes) * prod = 9/14 * 4/9 * 3/9 * 3/9 * 3/9 = 0.01058201
      P(Yes|x) = 1.000000   P(No|x) = 0.000000
      the other three factors for No were 1/5, 4/5, 3/5 -- none of them small.
```

A probability of exactly zero is a claim you cannot defend

The model is not saying "unlikely". It is saying **impossible**: no amount of evidence from the other three features can ever recover it, because they are all multiplied by 0. **One unobserved combination has vetoed the entire class.** And the sample size behind that veto is *five rows*.

Multiplication is unforgiving in a way that addition is not. This is the **zero-frequency problem**, and every naive Bayes implementation on earth ships with a fix turned on by default.

Laplace smoothing: pretend you saw one of everything

$$\hat{P}(x_j = t \mid c) \;=\; \frac{N_{tjc} + \alpha}{N_c + \alpha\, n_j}$$

where n_j is the number of levels feature j has, and $\alpha = 1$ is *Laplace* (add-one) smoothing. $\alpha < 1$ is called *Lidstone*.

`Outlook` has three levels, so for the *No* class the denominator becomes $5 + 3 = 8$:

$$P(\text{Overcast} \mid \text{No}) = \frac{0+1}{5+3} = \frac{1}{8} = 0.1250.$$

Add α to the top and αn_j to the bottom — so the column still sums to one. Getting the denominator wrong is the single commonest examination error on this topic.

The same instance, smoothed: before and after

```
the same instance with Laplace alpha = 1
      P(No) * prod = 5/14 * 1/8 * 2/8 * 5/7 * 4/7 = 0.00455539
      P(Yes) * prod = 9/14 * 5/12 * 4/12 * 4/11 * 4/11 = 0.01180638
      P(Overcast|No) = (0+1)/(5+3) = 0.1250
      P(Yes|x) = 0.721583   P(No|x) = 0.278417
      'impossible' has become 27.84%
```

The same instance, smoothed: before and after

```
the same instance with Laplace alpha = 1
      P(No) * prod = 5/14 * 1/8 * 2/8 * 5/7 * 4/7 = 0.00455539
      P(Yes) * prod = 9/14 * 5/12 * 4/12 * 4/11 * 4/11 = 0.01180638
      P(Overcast|No) = (0+1)/(5+3) = 0.1250
      P(Yes|x) = 0.721583   P(No|x) = 0.278417
      'impossible' has become 27.84%
```

	$P(\text{No} \mid x)$	$P(\text{Yes} \mid x)$
$\alpha = 0$	**0.000000**	1.000000
$\alpha = 1$	**0.278417**	0.721583

The same instance, smoothed: before and after

```
the same instance with Laplace alpha = 1
      P(No) * prod = 5/14 * 1/8 * 2/8 * 5/7 * 4/7 = 0.00455539
      P(Yes) * prod = 9/14 * 5/12 * 4/12 * 4/11 * 4/11 = 0.01180638
      P(Overcast|No) = (0+1)/(5+3) = 0.1250
      P(Yes|x) = 0.721583   P(No|x) = 0.278417
      'impossible' has become 27.84%
```

	$P(\text{No} \mid x)$	$P(\text{Yes} \mid x)$
$\alpha = 0$	**0.000000**	1.000000
$\alpha = 1$	**0.278417**	0.721583

Check the denominators against the level counts

`Outlook` and `Temperature` have three levels, so $5 + 3 = 8$. `Humidity` and `Wind` have two, so $5 + 2 = 7$. For *Yes*: $9 + 3 = 12$ and $9 + 2 = 11$. **Every fraction on that first line can be read straight off the level counts.**

How much smoothing? Sweep α and watch

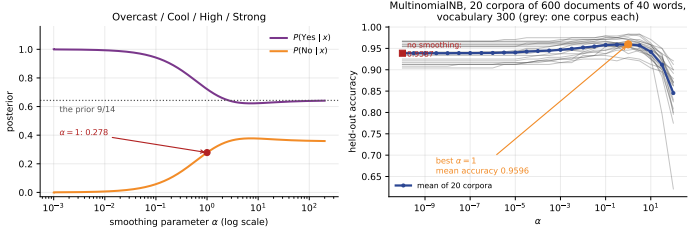
```
alpha sweep on the same instance
      alpha    P(No|x)   P(Yes|x)  prediction
       0.00   0.000000   1.000000  Yes
       0.01   0.006403   0.993597  Yes
       0.10   0.057749   0.942251  Yes
       1.00   0.278417   0.721583  Yes
       5.00   0.375650   0.624350  Yes
     100.00   0.360719   0.639281  Yes
   as alpha grows the conditionals wash out and the PRIOR takes over:
   P(Yes) = 9/14 = 0.6429, which is the limit above
```

How much smoothing? Sweep α and watch

```
alpha sweep on the same instance
      alpha    P(No|x)   P(Yes|x)  prediction
       0.00   0.000000   1.000000  Yes
       0.01   0.006403   0.993597  Yes
       0.10   0.057749   0.942251  Yes
       1.00   0.278417   0.721583  Yes
       5.00   0.375650   0.624350  Yes
     100.00   0.360719   0.639281  Yes
   as alpha grows the conditionals wash out and the PRIOR takes over:
   P(Yes) = 9/14 = 0.6429, which is the limit above
```

α interpolates between **trust the counts completely** ($\alpha \to 0$) and **ignore the features entirely** ($\alpha \to \infty$, where the posterior is the prior $9/14 = 0.6429$). It is a regularisation parameter, and like every regularisation parameter it is chosen by cross-validation.

Drawn: what α does, on a table and on text



Left: the tennis posterior as α sweeps from 0 to 200 — starting at the indefensible 0 and flattening onto the prior 9/14. Right: held-out accuracy of `MultinomialNB`, averaged over **twenty** corpora of 600 synthetic documents (grey: one corpus each — look how far one draw can wander). **No smoothing gives** 0.9387**; the best** α**, which is** 1**, gives** 0.9596, and too much costs more than too little: $\alpha = 100$ falls to 0.8454. α **is a hyperparameter, so cross-validate it — and not on one split.**

Reported honestly: on this table the label never flips

```
how many of the 36 possible days contain a zero, and how many flip?
   36 possible days; 12 of them make one class exactly impossible (33.3%)
   smoothing changes the prediction on 0 of the 36
mean |change| in P(Yes|x) across all 36 days when alpha goes 0 -> 1
   mean 0.0942   max 0.4356
```

Reported honestly: on this table the label never flips

```
how many of the 36 possible days contain a zero, and how many flip?
   36 possible days; 12 of them make one class exactly impossible (33.3%)
   smoothing changes the prediction on 0 of the 36
mean |change| in P(Yes|x) across all 36 days when alpha goes 0 -> 1
   mean 0.0942   max 0.4356
```

Say the honest thing, not the convenient thing

A third of all possible days drive one class to exactly zero. But the *argmax* never changes here, because every zero falls on *No* — the minority class, which was losing anyway. **What smoothing fixes on this table is the posterior, not the label**: on average it moves $P(\text{Yes} \mid x)$ by 0.0942 and at worst by 0.4356.

Reported honestly: on this table the label never flips

```
how many of the 36 possible days contain a zero, and how many flip?
   36 possible days; 12 of them make one class exactly impossible (33.3%)
   smoothing changes the prediction on 0 of the 36
mean |change| in P(Yes|x) across all 36 days when alpha goes 0 -> 1
   mean 0.0942   max 0.4356
```

Say the honest thing, not the convenient thing

A third of all possible days drive one class to exactly zero. But the *argmax* never changes here, because every zero falls on *No* — the minority class, which was losing anyway. **What smoothing fixes on this table is the posterior, not the label**: on average it moves $P(\text{Yes} \mid x)$ by 0.0942 and at worst by 0.4356.

On text the label does flip, because there the zeros land on both classes at once — which is the next slide.

A twelve-message spam corpus, built inline

```
SPAM = ["win a free laptop click this link now",
        "congratulations you win a free prize click now", ...]
HAM = ["the lecture notes for the machine learning class are on the portal",
        "please submit your assignment before the deadline on friday", ...]
```

```
12 messages (6 spam, 6 ham), vocabulary of 51 words
total word tokens: spam 46, ham 58
   word              n(spam)  n(ham)    P(w|S)    P(w|H)  log ratio
   free                    7       0   0.08247   0.00917     2.1961
   click                   6       0   0.07216   0.00917     2.0625
   win                     4       0   0.05155   0.00917     1.7261
   the                     1      11   0.02062   0.11009    -1.6751
   on                      0       3   0.01031   0.03670    -1.2697
   notes                   0       2   0.01031   0.02752    -0.9820
```

A twelve-message spam corpus, built inline

```
SPAM = ["win a free laptop click this link now",
        "congratulations you win a free prize click now", ...]
HAM = ["the lecture notes for the machine learning class are on the portal",
        "please submit your assignment before the deadline on friday", ...]
```

```
12 messages (6 spam, 6 ham), vocabulary of 51 words
total word tokens: spam 46, ham 58
   word              n(spam)  n(ham)    P(w|S)    P(w|H)  log ratio
   free                    7       0   0.08247   0.00917     2.1961
   click                   6       0   0.07216   0.00917     2.0625
   win                     4       0   0.05155   0.00917     1.7261
   the                     1      11   0.02062   0.11009    -1.6751
   on                      0       3   0.01031   0.03670    -1.2697
   notes                   0       2   0.01031   0.02752    -0.9820
```

With $\alpha = 1$ and $V = 51$: $P(\text{free} \mid S) = (7+1)/(46+51) = 8/97 = 0.08247$ and $P(\text{free} \mid H) = (0+1)/(58+51) = 1/109 = 0.00917$. **Every word becomes a weight, and classifying is adding them up.**

By hand: score "click the free link"

```
log P(S)  = log 0.5000 = -0.693147
log P(H)  = log 0.5000 = -0.693147
click    P(w|S)=0.07216 log=-2.6288   P(w|H)=0.00917 log=-4.6913
free     P(w|S)=0.08247 log=-2.4953   P(w|H)=0.00917 log=-4.6913
link     P(w|S)=0.04124 log=-3.1884   P(w|H)=0.00917 log=-4.6913
the      P(w|S)=0.02062 log=-3.8816   P(w|H)=0.11009 log=-2.2064
total log score  spam -12.887198   ham -16.973632
P(spam|msg) = 0.983479   P(ham|msg) = 0.016521
```

By hand: score "click the free link"

```
log P(S)  = log 0.5000 = -0.693147
log P(H)  = log 0.5000 = -0.693147
click    P(w|S)=0.07216 log=-2.6288   P(w|H)=0.00917 log=-4.6913
free     P(w|S)=0.08247 log=-2.4953   P(w|H)=0.00917 log=-4.6913
link     P(w|S)=0.04124 log=-3.1884   P(w|H)=0.00917 log=-4.6913
the      P(w|S)=0.02062 log=-3.8816   P(w|H)=0.11009 log=-2.2064
total log score  spam -12.887198   ham -16.973632
P(spam|msg) = 0.983479   P(ham|msg) = 0.016521
```

```
sklearn MultinomialNB(alpha=1)  ham 0.016521  spam 0.983479
agreement to 8.88e-16
```

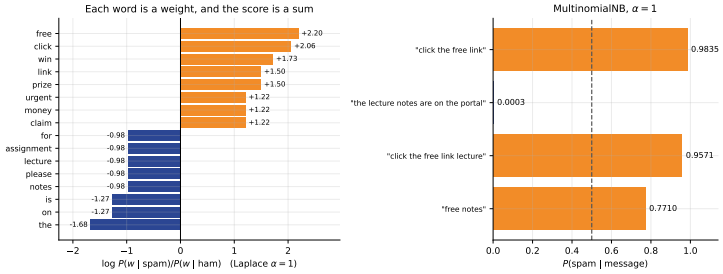
By hand: score "click the free link"

```
log P(S)  = log 0.5000 = -0.693147
log P(H)  = log 0.5000 = -0.693147
click    P(w|S)=0.07216 log=-2.6288   P(w|H)=0.00917 log=-4.6913
free     P(w|S)=0.08247 log=-2.4953   P(w|H)=0.00917 log=-4.6913
link     P(w|S)=0.04124 log=-3.1884   P(w|H)=0.00917 log=-4.6913
the      P(w|S)=0.02062 log=-3.8816   P(w|H)=0.11009 log=-2.2064
total log score  spam -12.887198   ham -16.973632
P(spam|msg) = 0.983479   P(ham|msg) = 0.016521
```

```
sklearn MultinomialNB(alpha=1)  ham 0.016521  spam 0.983479
agreement to 8.88e-16
```

Add five numbers, compare two totals. The margin is $-12.887198 - (-16.973632) = 4.086434$, and $1/(1 + e^{-4.086434}) = 0.983479$. **"The" argues for ham and loses, three to one.**

Drawn: every word is a weight



Left: $\log P(w \mid S)/P(w \mid H)$ for the sixteen most opinionated words. Right: four messages scored. **"Click the free link lecture" still scores** 0.9571 — one ham word cannot overturn three spam words, which is exactly what *summing* evidence should do, and exactly what a raw product cannot do.

Why we took logs at all: watch it underflow

Synthetic text: 600 documents, vocabulary 400, two classes sharing 88% of their vocabulary. Per-word probabilities run from 3.6×10^{-5} to 2.4×10^{-2}.

Predict first #4: multiply them one token at a time. At how many tokens does the running product become exactly 0.0**?**

Why we took logs at all: watch it underflow

Synthetic text: 600 documents, vocabulary 400, two classes sharing 88% of their vocabulary. Per-word probabilities run from 3.6×10^{-5} to 2.4×10^{-2}.

Predict first #4: multiply them one token at a time. At how many tokens does the running product become exactly 0.0?

tokens	raw product	sum of logs	exp(sum)
1	6.868132e-03	-4.9809	6.868132e-03
20	2.625046e-41	-93.4409	2.625046e-41
100	7.423356e-226	-518.3796	7.423356e-226
120	2.511950e-270	-620.7769	2.511950e-270
200	0.000000e+00	-1023.0046	0.000000e+00
400	0.000000e+00	-2050.9589	0.000000e+00

the raw product first becomes EXACTLY 0.0 at 148 tokens

Why we took logs at all: watch it underflow

Synthetic text: 600 documents, vocabulary 400, two classes sharing 88% of their vocabulary. Per-word probabilities run from 3.6×10^{-5} to 2.4×10^{-2}.

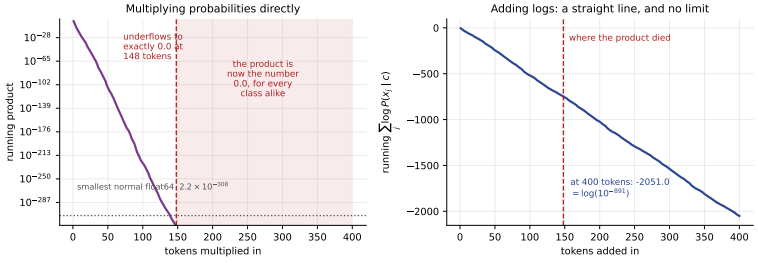
Predict first #4: multiply them one token at a time. At how many tokens does the running product become exactly 0.0?

```
   tokens      raw product      sum of logs       exp(sum)
        1     6.868132e-03          -4.9809   6.868132e-03
       20     2.625046e-41         -93.4409   2.625046e-41
      100    7.423356e-226        -518.3796  7.423356e-226
      120    2.511950e-270        -620.7769  2.511950e-270
      200     0.000000e+00       -1023.0046   0.000000e+00
      400     0.000000e+00       -2050.9589   0.000000e+00
the raw product first becomes EXACTLY 0.0 at 148 tokens
```

148 tokens

About one short paragraph. A real e-mail, a review, a support ticket — **all longer than that**. The smallest positive `float64` is 4.9×10^{-324}, and a product of 148 numbers averaging 10^{-3} walks straight past it.

Drawn: the product dies, the sum does not



Left: the running product on a log axis, falling off the bottom of `float64` at token 148. Right: the same computation as a sum of logs — **a straight line with no floor at all**. At 400 tokens it reads -2051.0, which is $\log(10^{-891})$: a number no floating-point format can hold, computed without difficulty.

And it is not a cosmetic problem

Score the whole test set twice — once by multiplying, once by adding logs. Documents are lengthened by tiling, the way an e-mail thread grows.

tokens	both scores 0.0	one score 0.0	raw-product acc	log acc
120	0	0	0.9722	0.9722
240	180	0	0.5056	0.9722
360	180	0	0.5056	0.9722
480	180	0	0.5056	0.9722

And it is not a cosmetic problem

Score the whole test set twice — once by multiplying, once by adding logs. Documents are lengthened by tiling, the way an e-mail thread grows.

tokens	both scores 0.0	one score 0.0	raw-product acc	log acc
120	0	0	0.9722	0.9722
240	180	0	0.5056	0.9722
360	180	0	0.5056	0.9722
480	180	0	0.5056	0.9722

0.9722 → 0.5056, and not one error message

At 240 tokens, **all** 180 **test documents** have both class scores equal to 0.0. The comparison `0.0 > 0.0` is `False`, so every one of them is silently assigned to class 0. **The accuracy halves and the program does not complain.**

And it is not a cosmetic problem

Score the whole test set twice — once by multiplying, once by adding logs. Documents are lengthened by tiling, the way an e-mail thread grows.

tokens	both scores 0.0	one score 0.0	raw-product acc	log acc
120	0	0	0.9722	0.9722
240	180	0	0.5056	0.9722
360	180	0	0.5056	0.9722
480	180	0	0.5056	0.9722

0.9722 → 0.5056, and not one error message

At 240 tokens, **all** 180 **test documents** have both class scores equal to 0.0. The comparison `0.0 > 0.0` is `False`, so every one of them is silently assigned to class 0. **The accuracy halves and the program does not complain.**

This is why `predict_log_proba` exists, why sklearn's naive Bayes classes store `feature_log_prob_` rather than probabilities, and why you should be suspicious of any product of more than about a hundred probabilities.

Getting back to a probability: log-sum-exp

The argmax needs only the log scores. If you want a posterior that sums to one, you must exponentiate — carefully.

```
logs [-800. -802.]
naive  exp(a).sum() = 0.0000e+00  -> log = -inf
stable mx + log(sum(exp(a-mx))) = -799.873072
```

Getting back to a probability: log-sum-exp

The argmax needs only the log scores. If you want a posterior that sums to one, you must exponentiate — carefully.

```
logs [-800. -802.]
naive  exp(a).sum() = 0.0000e+00  -> log = -inf
stable mx + log(sum(exp(a-mx))) = -799.873072
```

$$\log \sum_k e^{a_k} \;=\; a_{\max} + \log \sum_k e^{a_k - a_{\max}}$$

Subtracting the maximum makes the largest exponent exactly $e^0 = 1$, so nothing overflows and the biggest term never underflows. **The identity is exact — it is just** $\log(e^{a_{\max}} \sum e^{a - a_{\max}})$**.**

`scipy.special.logsumexp` does this for you, and it is the same trick that makes `softmax` numerically safe in every neural-network library. You will meet it again in Lecture 17.

What do you count when the feature is a real number?

You cannot

Petal width $= 1.7\,\text{cm}$ appears in the training set exactly zero times, or once. Counting gives you 0 or $1/50$, and neither is useful.

So model the density instead

$$P(x_j \mid c) \;=\; \frac{1}{\sqrt{2\pi\sigma_{jc}^2}} \exp\left(-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2}\right)$$

One mean and one variance per feature per class, both estimated by maximum likelihood — that is, the ordinary sample mean and the ordinary (biased, $\div N$) sample variance.

Two parameters replace a whole histogram. And note $P(x_j \mid c)$ is now a *density*, which can exceed 1; that is legal, and it normalises away when you divide by the evidence.

By hand: one iris flower, one measurement

```
load_iris, petal width (cm), by class
   class          n      mean   sd (ML)
   setosa        50    0.2460    0.1043
   versicolor    50    1.3260    0.1958
   virginica     50    2.0260    0.2719
```

A flower with petal width 1.7 cm. Substitute into the density:

```
      p(x|setosa     ) = 2.533257e-42   P(setosa|x) = 0.000000
      p(x|versicolor ) = 3.285675e-01   P(versicolor|x) = 0.314834
      p(x|virginica  ) = 7.150527e-01   P(virginica|x) = 0.685166
      PREDICTION: virginica
```

By hand: one iris flower, one measurement

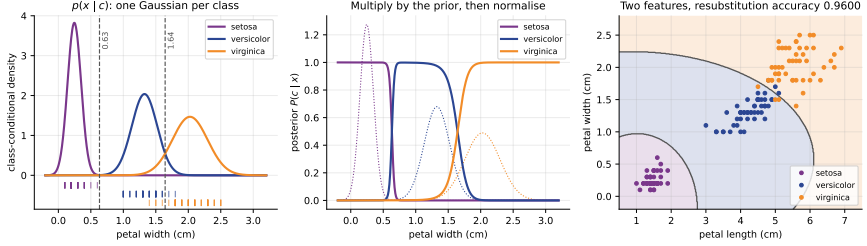
```
load_iris, petal width (cm), by class
   class          n      mean   sd (ML)
   setosa        50    0.2460    0.1043
   versicolor    50    1.3260    0.1958
   virginica     50    2.0260    0.2719
```

A flower with petal width 1.7 cm. Substitute into the density:

```
      p(x|setosa     ) = 2.533257e-42   P(setosa|x) = 0.000000
      p(x|versicolor ) = 3.285675e-01   P(versicolor|x) = 0.314834
      p(x|virginica  ) = 7.150527e-01   P(virginica|x) = 0.685166
      PREDICTION: virginica
```

Priors are equal (50 of each), so the posterior is just the densities normalised: $0.7150527/(0.3285675 + 0.7150527 + \text{tiny}) = 0.685166$. **And** `GaussianNB().predict_proba([[1.7]])` **returns** `[0.  0.3148 0.6852]`.

Drawn: three Gaussians and the boundaries they imply



Left: one fitted Gaussian per class over petal width, with the raw measurements as ticks below. The two decision boundaries sit at 0.6333 **and** 1.6437 **cm** — solved numerically, exactly where the curves cross once the priors are applied. Centre: the posteriors. Right: two features, resubstitution accuracy 0.9600.

How good is it? Measured against two harder models

GaussianNB against two others, 5-fold, on four bundled datasets

dataset	GaussianNB	LogReg	RandForest	fit ms NB	fit ms LR
iris	0.9600	0.9600	0.9400	0.45	1.69
wine	0.9719	0.9832	0.9719	0.41	1.82
breast_cancer	0.9385	0.9789	0.9649	0.53	2.40
digits	0.8508	0.9694	0.9733	1.29	128.02

dataset	rows x cols	NB passes	LR L-BFGS iterations
iris	150 x 4	1	17
wine	178 x 13	1	15
breast_cancer	569 x 30	1	19
digits	1797 x 64	1	32

How good is it? Measured against two harder models

GaussianNB against two others, 5-fold, on four bundled datasets

dataset	GaussianNB	LogReg	RandForest	fit ms NB	fit ms LR
iris	0.9600	0.9600	0.9400	0.45	1.69
wine	0.9719	0.9832	0.9719	0.41	1.82
breast_cancer	0.9385	0.9789	0.9649	0.53	2.40
digits	0.8508	0.9694	0.9733	1.29	128.02

dataset	rows x cols	NB passes	LR L-BFGS iterations
iris	150 x 4	1	17
wine	178 x 13	1	15
breast_cancer	569 x 30	1	19
digits	1797 x 64	1	32

Competitive, not best

It ties logistic regression on `iris`, matches the random forest on `wine`, and gives away four points on `breast_cancer` and twelve on `digits` — where neighbouring pixels are heavily dependent.

Quote the passes, not the milliseconds

One pass against 32 **L-BFGS iterations** on `digits`. Naive Bayes has *no* iterative solver: it reads the data once, accumulates sums and sums of squares, and stops. The `fit ms` columns

Predict first #5

`breast_cancer`, 60/40 split. `GaussianNB` scores 0.9342 on the held-out patients; logistic regression scores 0.9649.

Take every prediction naive Bayes makes and read off how confident it says it is. What is the average of those confidences?

Predict first #5

`breast_cancer`, 60/40 split. `GaussianNB` scores 0.9342 on the held-out patients; logistic regression scores 0.9649.

Take every prediction naive Bayes makes and read off how confident it says it is. What is the average of those confidences?

```
GaussianNB           fraction with max-probability > 0.99 : 0.9649
                     fraction with max-probability > 0.999: 0.9298
                     fraction in [0.1, 0.9]              : 0.0219
                     mean confidence 0.9936 against accuracy 0.9342
LogisticRegression   fraction with max-probability > 0.99 : 0.7105
                     fraction with max-probability > 0.999: 0.4956
                     fraction in [0.1, 0.9]              : 0.1360
                     mean confidence 0.9626 against accuracy 0.9649
```

Predict first #5

`breast_cancer`, 60/40 split. `GaussianNB` scores 0.9342 on the held-out patients; logistic regression scores 0.9649.

Take every prediction naive Bayes makes and read off how confident it says it is. What is the average of those confidences?

```
GaussianNB           fraction with max-probability > 0.99 : 0.9649
                     fraction with max-probability > 0.999: 0.9298
                     fraction in [0.1, 0.9]               : 0.0219
                     mean confidence 0.9936 against accuracy 0.9342
LogisticRegression   fraction with max-probability > 0.99 : 0.7105
                     fraction with max-probability > 0.999: 0.4956
                     fraction in [0.1, 0.9]               : 0.1360
                     mean confidence 0.9626 against accuracy 0.9649
```

0.9936 confident, **0.9342** right

93% of its answers claim a probability above 0.999, and only 2% of them land anywhere between 0.1 and 0.9. Logistic regression's mean confidence, 0.9626, sits essentially on top of its accuracy, 0.9649. **One of these models knows what it does not know.**

Where the claims and the reality part company

```
reliability: bin the predictions and compare claim with reality
   GaussianNB
      bin               n    mean p    actual      gap
      [0.00,0.10)      83    0.0014    0.0964  -0.0950
      [0.99,1.00)     139    0.9998    0.9496  +0.0501
   LogisticRegression
      bin               n    mean p    actual      gap
      [0.00,0.10)      75    0.0036    0.0000  +0.0036
      [0.99,1.00)      93    0.9980    1.0000  -0.0020
```

Where the claims and the reality part company

```
reliability: bin the predictions and compare claim with reality
   GaussianNB
      bin               n    mean p    actual      gap
      [0.00,0.10)      83    0.0014    0.0964  -0.0950
      [0.99,1.00)     139    0.9998    0.9496  +0.0501
   LogisticRegression
      bin               n    mean p    actual      gap
      [0.00,0.10)      75    0.0036    0.0000  +0.0036
      [0.99,1.00)      93    0.9980    1.0000  -0.0020
```

Read the first row

Eighty-three patients that naive Bayes called malignant with probability 0.9986. **Nearly one in ten of them was benign.** And of the 139 it called benign at 0.9998, five percent were not. **Its ranking is good; its numbers are fiction.**

Where the claims and the reality part company

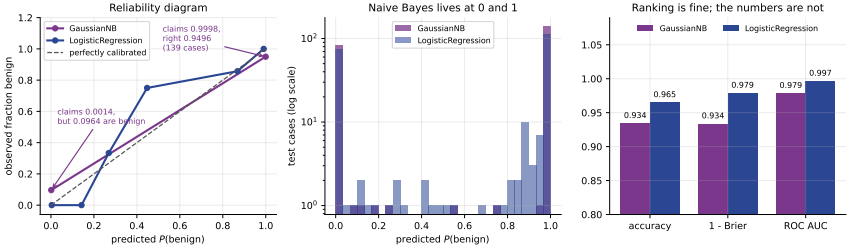
```
reliability: bin the predictions and compare claim with reality
   GaussianNB
      bin               n    mean p    actual      gap
      [0.00,0.10)      83    0.0014    0.0964  -0.0950
      [0.99,1.00)     139    0.9998    0.9496  +0.0501
   LogisticRegression
      bin               n    mean p    actual      gap
      [0.00,0.10)      75    0.0036    0.0000  +0.0036
      [0.99,1.00)      93    0.9980    1.0000  -0.0020
```

Read the first row

Eighty-three patients that naive Bayes called malignant with probability 0.9986. **Nearly one in ten of them was benign.** And of the 139 it called benign at 0.9998, five percent were not. **Its ranking is good; its numbers are fiction.**

The mechanism: independence makes the model count correlated evidence several times over, so the log-odds are inflated and the sigmoid saturates. We measure exactly that in the next section.

Drawn: naive Bayes lives at 0 and 1



Left: the reliability diagram — the purple curve should lie on the diagonal and does not. Centre: the histogram of predicted probabilities; naive Bayes has *two* bars and almost nothing in between. Right: the same model scored three ways. **Its ROC AUC is** 0.979 — **the ranking is nearly as good as logistic regression's** 0.997 — **while its Brier score is three times worse.**

Can it be repaired? Yes, afterwards

model	accuracy	Brier	AUC
GaussianNB, raw	0.9342	0.0663	0.9785
GaussianNB + sigmoid	0.9342	0.0626	0.9751
GaussianNB + isotonic	0.9211	0.0567	0.9646

Can it be repaired? Yes, afterwards

model	accuracy	Brier	AUC
GaussianNB, raw	0.9342	0.0663	0.9785
GaussianNB + sigmoid	0.9342	0.0626	0.9751
GaussianNB + isotonic	0.9211	0.0567	0.9646

What calibration is

`CalibratedClassifierCV` fits a second, monotone model that maps the raw scores to honest probabilities — a sigmoid (Platt scaling) or a step function (isotonic).

Reported honestly

The Brier score improves both times, 0.0663 to 0.0626 and 0.0567. The AUC does *not*: the wrapper averages five fold-models each fitted on 80% of only 341 rows, and isotonic reorders ties. **Calibration repairs the numbers, and on a small sample it costs a little ranking.**

Practical rule: **use** `predict` **freely, treat** `predict_proba` **with suspicion**, and calibrate before you use a naive Bayes probability to make a decision with a cost attached.

Copy one feature and watch the accuracy fall

Four genuinely independent informative features, classes separated by 1.6 standard deviations. Now add k near-copies of feature 0.

```
   copies of f0    d  GaussianNB    LogReg  RandForest
              0    4      0.9475    0.9467      0.9367
              1    5      0.9350    0.9483      0.9350
              3    7      0.9083    0.9467      0.9283
              7   11      0.8633    0.9458      0.9267
             15   19      0.8367    0.9467      0.9233
             31   35      0.8200    0.9483      0.9133
GaussianNB lost 0.1275 of accuracy; logistic regression lost -0.0017
```

Copy one feature and watch the accuracy fall

Four genuinely independent informative features, classes separated by 1.6 standard deviations. Now add k near-copies of feature 0.

```
 copies of f0    d  GaussianNB    LogReg  RandForest
            0    4      0.9475    0.9467      0.9367
            1    5      0.9350    0.9483      0.9350
            3    7      0.9083    0.9467      0.9283
            7   11      0.8633    0.9458      0.9267
           15   19      0.8367    0.9467      0.9233
           31   35      0.8200    0.9483      0.9133
GaussianNB lost 0.1275 of accuracy; logistic regression lost -0.0017
```

```
and what it does to the confidence
   k= 0  mean confidence 0.9457   accuracy 0.9556   fraction > 0.999 0.2833   Brier 0.0331
   k=31  mean confidence 0.9916   accuracy 0.8139   fraction > 0.999 0.9139   Brier 0.1802
```

Copy one feature and watch the accuracy fall

Four genuinely independent informative features, classes separated by 1.6 standard deviations. Now add k near-copies of feature 0.

```
 copies of f0   d  GaussianNB   LogReg  RandForest
            0   4      0.9475   0.9467      0.9367
            1   5      0.9350   0.9483      0.9350
            3   7      0.9083   0.9467      0.9283
            7  11      0.8633   0.9458      0.9267
           15  19      0.8367   0.9467      0.9233
           31  35      0.8200   0.9483      0.9133
GaussianNB lost 0.1275 of accuracy; logistic regression lost -0.0017
```

```
and what it does to the confidence
   k= 0  mean confidence 0.9457   accuracy 0.9556   fraction > 0.999 0.2833   Brier 0.0331
   k=31  mean confidence 0.9916   accuracy 0.8139   fraction > 0.999 0.9139   Brier 0.1802
```

0.9475 → 0.8200, on *no new information* — and it gets *more* certain

The added columns are copies: they contain nothing column 0 did not already contain. **Naive Bayes multiplies the same evidence thirty-two times**, because the product rule assumes each factor is a fresh observation. Logistic regression fits the columns jointly, splits the weight between them, and does not move at all. Meanwhile confidence rises 0.9457 → **0.9916** while accuracy falls 0.9556 → **0.8139**. **That is the mechanism behind the previous section.**

On real data: thirty columns that are ten measurements

`breast_cancer` records ten cell measurements three ways each — a mean, a standard error and a "worst" value.

```
mean |correlation| between the 30 columns: 0.3949
pairs with |r| > 0.9: 21 of 435
feature set                          d  GaussianNB    LogReg
all 30                              30      0.9385    0.9789
the 10 'mean' columns               10      0.9104    0.9455
the 10 'worst' columns              10      0.9456    0.9789
mean + worst (20)                   20      0.9420    0.9789
```

On real data: thirty columns that are ten measurements

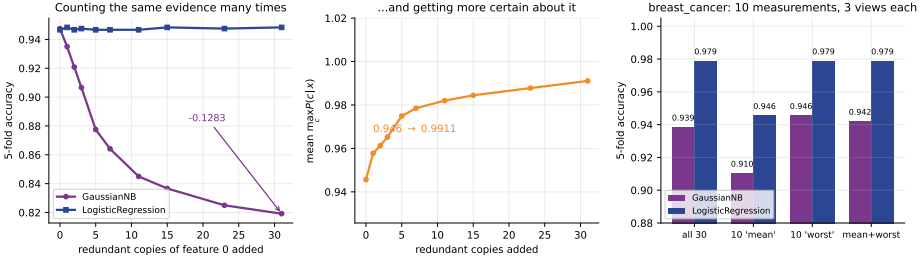
`breast_cancer` records ten cell measurements three ways each — a mean, a standard error and a "worst" value.

```
mean |correlation| between the 30 columns: 0.3949
pairs with |r| > 0.9: 21 of 435
feature set                          d  GaussianNB    LogReg
all 30                              30      0.9385    0.9789
the 10 'mean' columns               10      0.9104    0.9455
the 10 'worst' columns              10      0.9456    0.9789
mean + worst (20)                   20      0.9420    0.9789
```

Ten columns beat thirty: the `worst` block alone gives `GaussianNB` 0.9456 against 0.9385 for all thirty. **Dropping two-thirds of the data improved the model.**

So for naive Bayes, **de-duplicating correlated columns is a real modelling step**, not tidying. It is the one place where feature selection reliably pays for this algorithm.

Drawn: counting the same evidence many times



Left: accuracy against redundant copies — purple falls, blue does not move. Centre: mean confidence over the same sweep, rising from 0.9457 to 0.9911 as accuracy falls. Right: the real `breast_cancer` feature blocks.

A different failure: XOR

Two binary features, label $= u \oplus v$, plus a little noise. The four (u, v) cells hold 300 rows each *by construction*, so the symmetry is exact and not a lucky sample.

```
GaussianNB    0.4600
LogReg        0.4600
RandForest    0.9833
   class 0: mean u 0.5063  mean v 0.4974  corr(u,v|y) +0.8621
   class 1: mean u 0.4819  mean v 0.4936  corr(u,v|y) -0.8573
```

A different failure: XOR

Two binary features, label $= u \oplus v$, plus a little noise. The four (u, v) cells hold 300 rows each *by construction*, so the symmetry is exact and not a lucky sample.

```
GaussianNB    0.4600
LogReg        0.4600
RandForest    0.9833
   class 0: mean u 0.5063  mean v 0.4974  corr(u,v|y) +0.8621
   class 1: mean u 0.4819  mean v 0.4936  corr(u,v|y) -0.8573
```

Chance level, on a perfectly learnable problem

Read the last two lines. Within each class the means of u and v are both about 0.5: **each feature alone is completely uninformative**, so every per-feature factor $P(x_j \mid c)$ is identical for both classes and the product cannot distinguish them. All the signal lives in the *correlation*, which is $+0.86$ in one class and -0.86 in the other — precisely what the naive assumption throws away.

A different failure: XOR

Two binary features, label $= u \oplus v$, plus a little noise. The four (u, v) cells hold 300 rows each *by construction*, so the symmetry is exact and not a lucky sample.

```
GaussianNB    0.4600
LogReg        0.4600
RandForest    0.9833
   class 0: mean u 0.5063  mean v 0.4974  corr(u,v|y) +0.8621
   class 1: mean u 0.4819  mean v 0.4936  corr(u,v|y) -0.8573
```

Chance level, on a perfectly learnable problem

Read the last two lines. Within each class the means of u and v are both about 0.5: **each feature alone is completely uninformative**, so every per-feature factor $P(x_j \mid c)$ is identical for both classes and the product cannot distinguish them. All the signal lives in the *correlation*, which is $+0.86$ in one class and -0.86 in the other — precisely what the naive assumption throws away.

A random forest gets 0.9833. **When the information is in the interaction, use a model that can represent interactions.**

So why does anyone still use it? Because of this table

Fit both models on a shrinking training set, average over 30 random splits.

n train	GaussianNB	LogReg	RandForest
20	0.9271	0.9276	0.9161
40	0.9366	0.9502	0.9287
80	0.9374	0.9640	0.9393
200	0.9357	0.9711	0.9478
400	0.9385	0.9763	0.9548

So why does anyone still use it? Because of this table

Fit both models on a shrinking training set, average over 30 random splits.

n train	GaussianNB	LogReg	RandForest
20	0.9271	0.9276	0.9161
40	0.9366	0.9502	0.9287
80	0.9374	0.9640	0.9393
200	0.9357	0.9711	0.9478
400	0.9385	0.9763	0.9548

At $n = 20$ it is level with logistic regression

Twenty patients, thirty features. Naive Bayes estimates 30×2 means and 30×2 variances, each from ten rows, and scores 0.9271. **It has almost no variance to bleed, because it never fits a joint anything.** By $n = 400$ logistic regression is four points ahead — but you often do not have 400 rows.

So why does anyone still use it? Because of this table

Fit both models on a shrinking training set, average over 30 random splits.

n train	GaussianNB	LogReg	RandForest
20	0.9271	0.9276	0.9161
40	0.9366	0.9502	0.9287
80	0.9374	0.9640	0.9393
200	0.9357	0.9711	0.9478
400	0.9385	0.9763	0.9548

At $n = 20$ it is level with logistic regression

Twenty patients, thirty features. Naive Bayes estimates 30×2 means and 30×2 variances, each from ten rows, and scores 0.9271. **It has almost no variance to bleed, because it never fits a joint anything.** By $n = 400$ logistic regression is four points ahead — but you often do not have 400 rows.

Domingos and Pazzani's result, in a table: **a wrong probability model can still put the right class on top**, because the argmax only needs the ordering, not the values.

Summary — fifteen lines

1. Bayes' theorem is the product rule written twice and divided: $P(c \mid x) = P(x \mid c)P(c)/P(x)$. **Prior**, **likelihood**, **evidence**, **posterior**. It is a rearrangement, not an assumption.
2. The medical test: 1% prevalence, 99% sensitivity, 95% specificity. A positive result gives $P(D \mid +) = 99/594 = \mathbf{0.1667}$, not 0.99. **The base-rate fallacy is confusing** $P(D \mid +)$ **with** $P(+ \mid D)$.
3. In whole people: of 10,000, 594 test positive and 99 are ill. **False positives outnumber true positives five to one.** Two million simulated people agree to 0.001.
4. Specificity, not sensitivity, is the lever: 95% → 99.9% takes the posterior from 0.1667 to 0.9091. Raising prevalence to 20% takes it to 0.8319.
5. **ML** maximises $P(x \mid c)$, **MAP** maximises $P(x \mid c)P(c)$. On this evidence they disagree: ML says disease, MAP says no disease, because a likelihood ratio of 19.8 cannot overcome prior odds of 99.
6. Posterior odds = likelihood ratio × prior odds. One multiplication.
7. The full joint likelihood needs $C(2^d - 1)$ parameters: at $d = 30$ that is 2,147,483,646 against 569 training rows. It exceeds the data at $\mathbf{d = 9}$.
8. **The naive assumption**: $P(x_1, \ldots, x_d \mid c) = \prod_j P(x_j \mid c)$ — independence *given the class*. On the tennis table it takes 70 free parameters down to **12**.
9. By hand on (Sunny, Cool, High, Strong): *No* scores 0.02057143, *Yes* 0.00529101, so $P(\text{No} \mid x) = \mathbf{0.795417}$ — and `CategoricalNB` agrees to 7.68×10^{-12}.
10. **Zero frequency**: $P(\text{Overcast} \mid \text{No}) = 0/5$ makes the whole product 0. Laplace, $(0+1)/(5+3) = 0.125$, turns $P(\text{No} \mid x)$ from **0.000000** into **0.278417**. Twelve of the 36 possible days contain a zero; on this table the *label* never flips, but the posterior moves by up to 0.4356.
11. As $\alpha \to \infty$ the posterior tends to the prior $9/14 = 0.6429$. α is a regularisation parameter: cross-validate it.
12. **Logs.** A product of per-token probabilities hits exactly 0.0 at **148** tokens. Multiplying instead of adding logs took test accuracy from 0.9722 to **0.5056**, silently. Recover a posterior with log-sum-exp: $a_{\max} + \log \sum e^{a - a_{\max}}$.
13. **Gaussian NB** replaces counting with one mean and one variance per feature per class. Iris petal width 1.7 cm gives $P(\text{virginica}) = 0.685166$; boundaries at 0.6333 and 1.6437 cm. Fits `digits` in *one* pass against logistic regression's 32 iterations.
14. **Good classifier, bad probability estimator**: mean confidence **0.9936** against accuracy **0.9342**; 93% of predictions above 0.999; Brier three times worse than logistic regression while AUC is 0.979 against 0.997.
15. **Correlated features are the mechanism.** Thirty-one copies of one column took accuracy 0.9475 → 0.8200 and confidence 0.9457 → 0.9916. On `breast_cancer`, ten columns beat thirty. On XOR, naive Bayes scores 0.4600. But at $n = 20$ it ties logistic regression — which is why it survives.

Before the next lecture

Read

`notes.pdf` — the full development, the derivation from the product rule, the complete smoothing treatment, and **10 practice questions with worked solutions**.

Do

Questions 1, 2, 3 and 5, **with a pen**. Question 1 is a base-rate calculation; question 3 is a full naive Bayes by hand with Laplace smoothing. These are the numerically examinable ones.

Next: ensemble methods

Bagging, boosting and random forests — many weak models combined by vote. Carry one contrast across: **naive Bayes wins by making a strong wrong assumption and paying almost nothing for it; an ensemble wins by making no assumption and paying in computation.**

And a habit to start now

Any time you read a probability off a classifier, ask what it would take to check it. Bin the predictions, compare the claim with the outcome. **A model that is right** 93% **of the time and says** 99.4% **is lying to you politely.**

Materials: `https://github.com/tali7c/Applied-Machine-Learning`

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 8–9.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 8–9.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 4–5, 7, 14.; Zhang et al., *Dive into Deep Learning*, Ch. 5.